

Complete The Graph

Caminhos mínimos com
pesos desconhecidos e
Dijkstra

DISCIPLINA

Resolução de Problemas com Grafos

ALUNO

João Victor Feijó Vasconcelos

ORIENTADOR

Prof. Me. Ricardo Carubbi

Roteiro da apresentação

01

Problema e modelagem

O enunciado convertido em grafo e a representação adotada

02

Estratégia com Dijkstra

Monotonicidade, busca binária e reconstrução do caminho

03

Complexidade

Custo por execução do Dijkstra e custo total

04

Casos especiais e conclusão

Quando a resposta é NO e a evidência de Accepted

O problema

Um grafo **não-direcionado** tem algumas arestas com o **peso apagado** (valor 0).

Precisamos atribuir um peso inteiro positivo a cada aresta apagada de modo que o **menor caminho** entre a origem **s** e o destino **t** seja **exatamente L**.

RESTRIÇÕES

$$n \leq 1000$$

vértices

$$m \leq 10^4$$

arestas

$$L \leq 10^9$$

distância alvo

$$w \geq 1$$

peso atribuído

YES + pesos das arestas

NO se impossível

Entradas do Codeforces

```
# linha 1: parâmetros do grafo
n m L s t
# próximas m linhas: cada aresta
u v w (w = 0 → apagada)
```

O Dijkstra lê os dados uma única vez e guarda cada aresta por índice, acessando o peso original em tempo constante.

Exemplo 1

YES · aresta 1-4 = 8

```
5 5 13 0 4
0 1 5
2 1 2
3 2 3
1 4 0
4 3 4
```

Exemplo 2

YES · aresta = L

```
2 1 123456789 0 1
0 1 0
```

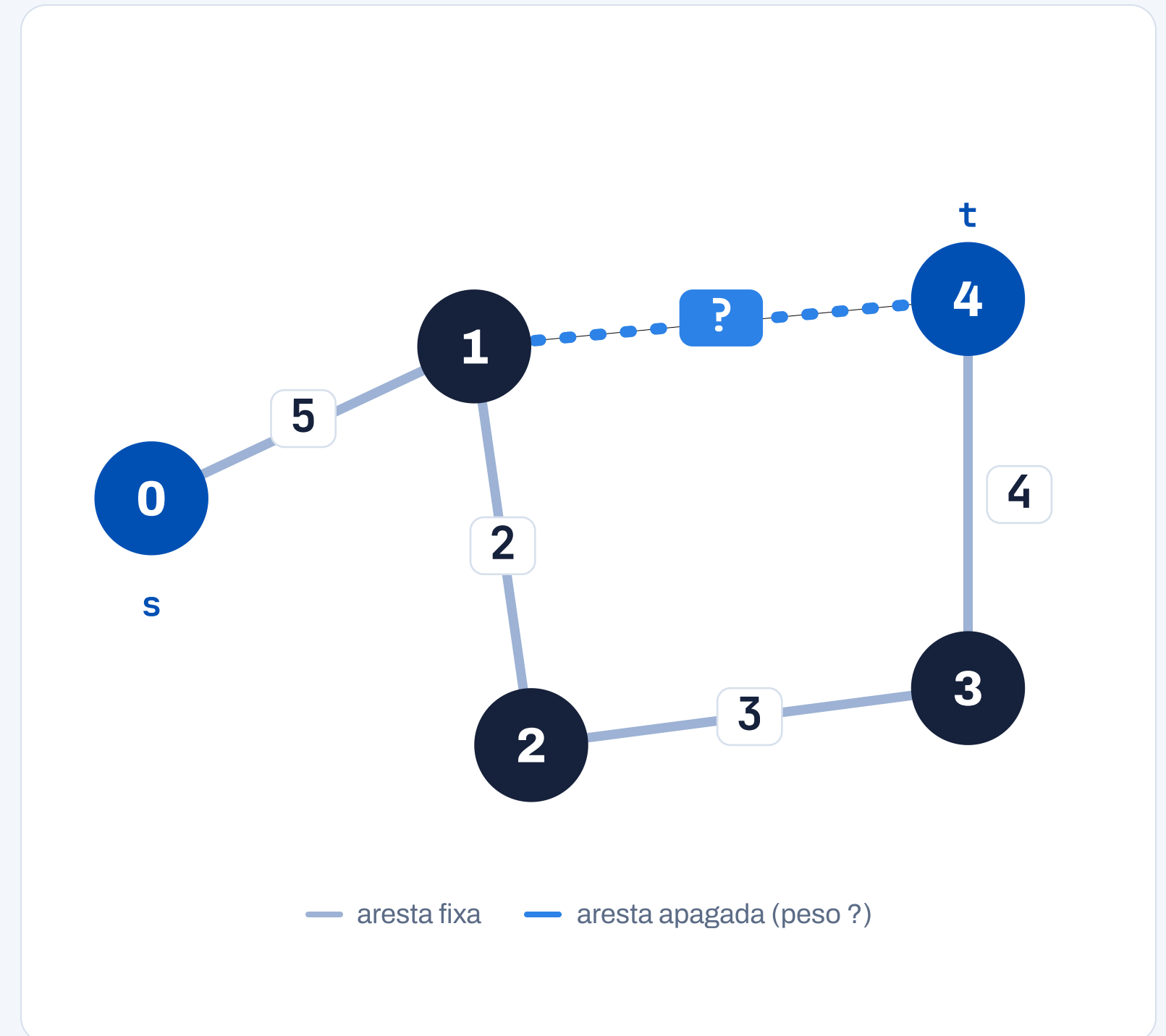
Exemplo 3

NO

```
2 1 999999999 1 0
0 1 1000000000
```

Modelagem como grafo

- Vértices** os n nós, numerados de 0 a $n-1$
- Arestas** m arestas não-direcionadas com peso; as de peso **0** estão apagadas
- Pesos** fixos quando dados; **incógnitas** (≥ 1) nas apagadas
- $s \rightarrow t$** origem e destino; objetivo é **$\text{dist}(s,t) = L$**
- Estrutura** **lista de adjacência** + vetor de arestas por índice $\rightarrow$ peso em $O(1)$

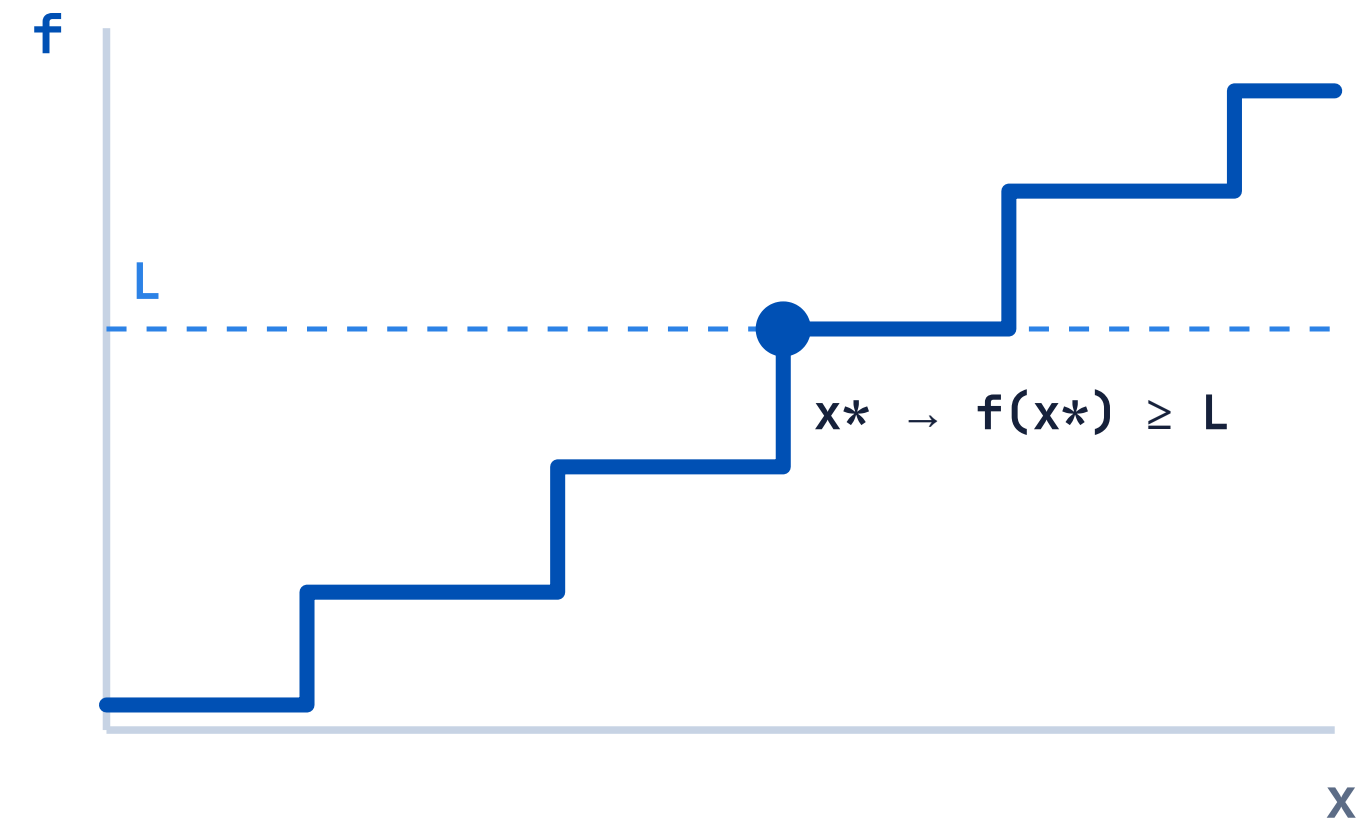


Ideia central: monotonicidade

Defina $f(x)$ = menor distância de s a t quando **toda** aresta apagada recebe o peso x .

Aumentar x só pode **manter ou aumentar** a distância: f é **não-decrescente**.

Logo podemos fazer **busca binária** em x para chegar perto de L .



O algoritmo de Dijkstra

- ◆ **Fila de prioridade mínima** (heapq) entrega sempre o vértice mais próximo ainda não fixado.
- ◆ **Relaxamento**: para cada vizinho, se $d[u] + \text{peso} < d[v]$, atualiza $d[v]$ e o predecessor.
- ◆ **Deleção preguiçosa**: entradas obsoletas na heap são ignoradas ao serem retiradas.
- ◆ **Predecessores** guardam de onde cada vértice foi alcançado → **reconstrução do caminho**.

```
# custo da aresta apagada vira parâmetro
w = arestas[idx][2] or val_zero
nova = d + w
if nova < dist[v]:
    dist[v] = nova
    anterior[v] = u
    heappush(fila, (nova, v))
```

Pesos não-negativos garantem que Dijkstra é aplicável — uma vez retirado da fila, o vértice tem distância definitiva.

Estratégia em três passos

1

Viabilidade

Testa os extremos. Se $f(1) > L$ ou $f(L+1) < L$, a resposta é **NO**.

Com peso mínimo já passa de L , ou as arestas fixas já dão menos que L .

2

Busca binária

Encontra o menor x^* em $[1, L+1]$ com $f(x^*) \geq L$.

~30 execuções de Dijkstra estreitam o intervalo até o ponto ideal.

3

Ajuste fino

Reconstrói o caminho e reduz $\delta = f(x^*)$
- L arestas de x^* para x^*-1 .

O caminho-alvo fica exatamente L ; os demais permanecem $\geq L$.

Análise de complexidade

◆ Por chamada de Dijkstra

OPERAÇÃO	CUSTO
Inserções na heap ($\leq$ uma por relaxamento)	$O(m)$
Cada inserção / remoção (heap com $\leq O(m)$ entradas)	$O(\log m) = O(\log n)$
Varredura das listas de adjacência	$O(n + m)$
Total por Dijkstra	$O((n + m) \log n)$

$\log m = O(\log n)$ pois $m \leq n^2 \rightarrow \log m \leq 2 \log n$

◆ Caso geral

CASO	CENÁRIO	COMPLEXIDADE
Melhor	Verificação de viabilidade falha ($d_{low} > L$ ou $d_{high} < L$) — apenas 2 chamadas de Dijkstra	$O((n + m) \log n)$
Médio / Pior	Busca binária completa (grafo esparsa ou denso)	$O(\log L \times (n + m) \log n)$

Com $n \leq 1000$, $m \leq 10^4$ e $L \leq 10^9$, são ~33 execuções de Dijkstra — bem dentro do limite de tempo do juiz.

Casos especiais

NO Mínimo grande demais

Mesmo com todas as apagadas valendo 1, $f(1) > L$. Não há como encurtar mais.

NO Caminho fixo curto

Com as apagadas "desligadas", $f(L+1) < L$: as arestas fixas já dão menos que L.

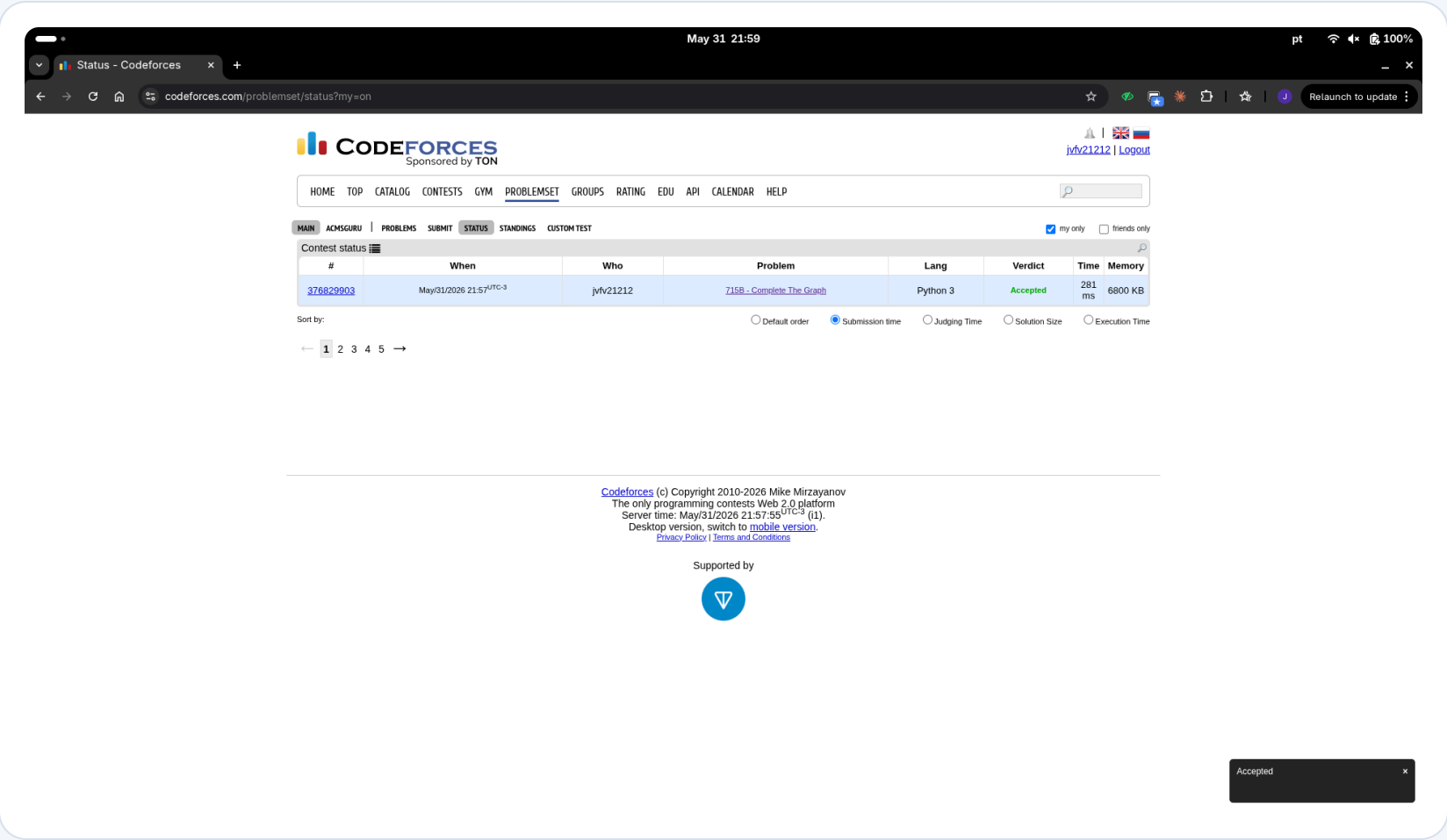
≥ 1 Peso sempre válido

Ao reduzir para $x*-1$, garante-se $x* \geq 2$ quando $\delta > 0$ — nunca gera peso zero.

10^9 Distâncias grandes

L chega a 10^9 e pesos somam mais; os inteiros nativos do Python evitam overflow.

Resultado e conclusão



Modelar como grafo e enxergar a **monotonicidade de $f(x)$** transforma "pesos desconhecidos" em uma **busca binária + Dijkstra**.

Accepted

281 ms

6800 KB

Python 3

Submissão 376829903 · Codeforces 715B — Complete The Graph.

Obrigado.

Perguntas são bem-vindas.

Problema `codeforces.com/problemset/problem/715/B`

Repositório `github.com/Jotave8714/und3atv2`

Aluno João Victor Feijó Vasconcelos